

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA5113 **COURSE NAME:** Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 50

Annexure A

Coding Challenge

Code of Conduct:

I Kartheeka Repalle (192372289) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Kartheeka Repalle

Annexure A

Coding Challenge

1. Write a program to simulate the IPSec Authentication Header (AH) process that generates and verifies a message authentication code (MAC) for IP packet integrity.
2. Develop a program to implement Encapsulating Security Payload (ESP) for encrypting and decrypting IP packet payloads.
3. Design a program to manage Security Associations (SA) in IPSec, including creation, lookup, and deletion operations.
4. Implement a simplified Pretty Good Privacy (PGP) system for secure email communication using public key encryption and digital signatures.
5. Create a program to classify emails as spam or legitimate using email header analysis and content-based filtering techniques.

Answers:

1. IPSec Authentication Header (AH) Simulation

Code

```
import hmac
import hashlib

key = b"secretkey"
packet = b"HELLO IP PACKET"

mac = hmac.new(key, packet, hashlib.sha256).hexdigest()

print("IPSec AH Simulation")
print("Packet :", packet.decode())
print("AH MAC :", mac)
```

```
verify = hmac.new(key, packet, hashlib.sha256).hexdigest()
```

```
if mac == verify:
```

```
    print("Verification : SUCCESS")
```

```
else:
```

```
    print("Verification : FAILED")
```

Output

IPSec AH Simulation

Packet : HELLO IP PACKET

AH MAC : 8f.....

Verification : SUCCESS

2. IPSec Encapsulating Security Payload (ESP)

Code

```
from cryptography.fernet import Fernet
```

```
key = Fernet.generate_key()
```

```
cipher = Fernet(key)
```

```
data = b"HELLO IP PACKET"
```

```
encrypted = cipher.encrypt(data)
```

```
decrypted = cipher.decrypt(encrypted)
```

```
print("IPSec ESP Simulation")
```

```
print("Original :", data.decode())
```

```
print("Encrypted :", encrypted.decode())
```

```
print("Decrypted :", decrypted.decode())
```

```
if data == decrypted:
    print("Verification : SUCCESS")
```

Output

```
IPSec ESP Simulation
Original : HELLO IP PACKET
Encrypted : gAAAAAB.....
Decrypted : HELLO IP PACKET
Verification : SUCCESS
```

3. IPSec Security Association (SA) Management

Code

```
SA = {}

# Create
SA["SA1"] = {
    "Source": "192.168.1.10",
    "Destination": "192.168.1.20",
    "SPI": 1001,
    "Algorithm": "AES"
}

print("SA Created")
print("SA Details:", SA["SA1"])

# Lookup
if "SA1" in SA:
    print("Lookup : SA Found")

# Delete
del SA["SA1"]
```

```
print("Delete : SA Deleted")
```

```
if "SA1" not in SA:
```

```
    print("Lookup after deletion : SA Not Found")
```

Output

SA Created

SA Details: {'Source': '192.168.1.10', 'Destination': '192.168.1.20', 'SPI': 1001, 'Algorithm': 'AES'}

Lookup : SA Found

Delete : SA Deleted

Lookup after deletion : SA Not Found

4. Simplified PGP System

Code

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
```

```
# Generate keys
```

```
private = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
```

```
public = private.public_key()
```

```
message = b"Secure Email"
```

```
# Sign message
```

```
signature = private.sign(
    message,
```

```

padding.PSS(
    mgf=padding.MGF1(hashes.SHA256()),
    salt_length=padding.PSS.MAX_LENGTH
),
hashes.SHA256()
)

# Verify signature
try:
    public.verify(
        signature,
        message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
    print("PGP Simulation")
    print("Message :", message.decode())
    print("Signature : Valid")
except:
    print("Signature : Invalid")

```

Output

```

PGP Simulation
Message : Secure Email
Signature : Valid

```

5. Email Spam Classification

Code

```
emails = [  
    ("winner@unknown.com", "You won a FREE prize"),  
    ("teacher@college.edu", "Submit your assignment"),  
    ("offer@shop.com", "FREE discount offer"),  
    ("friend@gmail.com", "Meeting tomorrow")  
]  
  
spam_words = ["free", "winner", "prize", "offer", "discount"]  
  
print("Email Spam Classification")  
  
for sender, subject in emails:  
  
    text = (sender + " " + subject).lower()  
    score = 0  
  
    for word in spam_words:  
        if word in text:  
            score += 1  
  
    if score >= 2:  
        result = "SPAM"  
    else:  
        result = "LEGITIMATE"  
  
    print("\nSender :", sender)  
    print("Subject:", subject)  
    print("Score :", score)  
    print("Result :", result)
```

Output

Email Spam Classification

Sender : winner@unknown.com

Subject: You won a FREE prize

Score : 3

Result : SPAM

Sender : teacher@college.edu

Subject: Submit your assignment

Score : 0

Result : LEGITIMATE

Sender : offer@shop.com

Subject: FREE discount offer

Score : 3

Result : SPAM

Sender : friend@gmail.com

Subject: Meeting tomorrow

Score : 0

Result : LEGITIMATE